# Python–MySQL Connectivity

## 1. Introduction

Connecting Python with MySQL allows us to build database-driven applications. The `mysql.connector` module acts as a bridge between Python and the MySQL database server.

---

## 2. Steps to Connect Python and SQL

① Import → ② connect() → ③ cursor() → ④ execute()
→ ⑤ fetch* / commit → ⑥ close()

### Step 1: Import the module

```
import mysql.connector as sql
```

You can use `sql` as an alias for convenience.

### Step 2: Establish connection using `connect()`

The `connect()` function requires **4 parameters** (3 compulsory, 1 optional):

| Parameter | Description | Compulsory / Optional |
|---|---|---|
| `host` | Server address (usually `"localhost"`) | Compulsory |
| `user` | MySQL username (default: `"root"`) | Compulsory |

| Parameter | Description | Compulsory / Optional |
|---|---|---|
| `password` or `passwd` | MySQL password | Compulsory |
| `database` | Name of the database to connect to | Optional |

```
con = sql.connect(
    host = "localhost",
    user = "root",
    passwd = "mysql123",
    database = "ip12"
)
```

✓ **Key points:**
• Only **3 parameters are compulsory** (host, user, password/passwd).
• **1 parameter is optional** (database) – if omitted, you connect without selecting a database.
• You can use either `password` or `passwd`.

**Verify connection:**

```
if con.is_connected():
    print("Connected successfully")
else:
    print("Connection failed")
```

## Step 3: Create a cursor using `cursor()`

A **cursor** enables Python to execute SQL commands and fetch results.

```
cur = con.cursor()  # no parameters
```

- Acts as a pointer to traverse database records.
- Executes SQL queries and fetches data.

## Step 4: Execute SQL query using `execute()`

```
cur.execute("SQL QUERY")
```

### DDL queries (Data Definition Language)

```
cur.execute("CREATE TABLE students (rollno INT PRIMARY KEY, name
VARCHAR(30), age INT)")
cur.execute("ALTER TABLE students ADD COLUMN city VARCHAR(20)")
```

### DML queries (Data Manipulation Language)

```
-- SELECT
cur.execute("SELECT * FROM Student;")

-- INSERT
cur.execute("INSERT INTO students VALUES (101, 'Geeta', 20)")

-- UPDATE
cur.execute("UPDATE students SET city = 'Delhi' WHERE rollno = 101")

-- DELETE
cur.execute("DELETE FROM students WHERE rollno = 101")
```

## Step 5: Fetch results for SELECT queries

> 🔑 **Important distinction:**
> **Retrieving** = data moves from SQL → Cursor
> **Fetching** = data moves from Cursor → Python (variable / print)

| Function | Description | Return Type |
|---|---|---|
| `fetchall()` | Fetches **all rows** retrieved | List of tuples |
| `fetchmany(n)` | Fetches only **'n' number of rows** | List of tuples |
| `fetchone()` | Fetches **only one row** | Tuple |

### fetchall() – all rows

```
cur.execute("SELECT * FROM Student;")
data = cur.fetchall()  # list of tuples
for row in data:
    print(row)
```

### fetchmany(n) – first n rows

```
cur.execute("SELECT * FROM Student;")
data = cur.fetchmany(5)  # first 5 rows
for row in data:
    print(row)
```

### fetchone() – one row at a time

```
cur.execute("SELECT * FROM Student;")
row = cur.fetchone()  # single tuple
while row is not None:
    print(row)
    row = cur.fetchone()
```

### Using `rowcount`

`cur.rowcount` gives the number of rows retrieved/affected by the last query.

```
cur.execute("SELECT * FROM Student;")
cur.fetchall()
print("Total rows retrieved:", cur.rowcount)
```

> ℹ️ `rowcount` works with both SELECT queries (rows retrieved) and DML queries (rows affected).

## Step 6: Commit changes for DML queries

For **INSERT, UPDATE, DELETE** queries, changes are *not* automatically saved. You must call `commit()`.

```
con.commit()
```

> ⚠️ **Without** `commit()`, DML changes are temporary and lost when the connection closes.

```
-- Update example
cur.execute("UPDATE students SET city = 'Mumbai' WHERE rollno = 101")
con.commit()  # MUST commit

-- Delete example
cur.execute("DELETE FROM students WHERE rollno = 102")
con.commit()  # MUST commit

-- Insert example
cur.execute("INSERT INTO students VALUES (103, 'Raj', 18, 'Delhi')")
con.commit()  # MUST commit
```

### Step 7: Close the connection

```
cur.close()
con.close()
```

Always release resources to avoid memory leaks.

---

## 3. Complete Workflow Diagram

```
① import  mysql.connector as sql
② connect() → con = sql.connect(host, user, passwd, database)
③ cursor() → cur = con.cursor()
④ execute() → cur.execute("SQL query")
   ├── SELECT → ⑤ fetchall() / fetchone() / fetchmany(n)
   └── INSERT/UPDATE/DELETE → ⑥ commit() → con.commit()
```

⑦ close() → cur.close(); con.close()

---

## 4. Creating a Database

Before connecting to a specific database, you may need to **create a new database**. This is done using a `CREATE DATABASE` SQL query.

> 📌 **Important:** To create a database, you must connect **without specifying a database** in the `connect()` function.

### Step-by-step: Create a new database

```
import mysql.connector as sql

# Step 1: Connect WITHOUT specifying a database
con = sql.connect(
    host="localhost",
    user="root",
    passwd="mysql123"
    # No 'database' parameter here
)

cur = con.cursor()

# Step 2: Execute CREATE DATABASE query
cur.execute("CREATE DATABASE IF NOT EXISTS school_db")

# Step 3: Verify database creation
cur.execute("SHOW DATABASES")
databases = cur.fetchall()
print("Databases:")
for db in databases:
    print(db[0])

# Step 4: Now connect to the newly created database
con.database = "school_db"  # Switch to new database
```

```
# Step 5: Create tables in the new database
cur.execute("""CREATE TABLE IF NOT EXISTS students (
    rollno INT PRIMARY KEY,
    name VARCHAR(30),
    age INT,
    city VARCHAR(20)
)""")

print("Database 'school_db' created and table 'students' created
successfully!")

cur.close()
con.close()
```

### Alternative: Connect directly to the new database

```
# After creating the database, you can connect directly
con = sql.connect(
    host="localhost",
    user="root",
    passwd="mysql123",
    database="school_db"  # Now specify the newly created database
)
```

**✓ Key points for creating a database:**

- Connect **without** the `database` parameter to create a new database.
- Use `CREATE DATABASE IF NOT EXISTS` to avoid errors if the database already exists.
- After creation, you can set the database using `con.database = "db_name"`.
- You can also reconnect with the `database` parameter specified.

---

## 5. Parameterized Queries: `%s` and `format()`

Parameterized queries allow you to insert **dynamic values** into SQL statements safely. This prevents SQL injection attacks and makes your code cleaner.

## 5.1 Using `%s` Format Specifier

The `%s` acts as a placeholder for values. You pass the actual values as a **tuple** to the `execute()` method.

> ⚠️ **Security Note:** Using `%s` placeholders is the **recommended** and **safer** way to execute queries with dynamic values.

### INSERT with `%s`

```
# Single row insertion
sql = "INSERT INTO students (rollno, name, age, city) VALUES (%s, %s,
%s, %s)"
values = (101, "Geeta Sharma", 20, "Kolkata")
cur.execute(sql, values)  # Pass values as tuple
con.commit()
print(cur.rowcount, "record(s) inserted")
```

### UPDATE with `%s`

```
sql = "UPDATE students SET city = %s WHERE rollno = %s"
cur.execute(sql, ("Mumbai", 101))  # Tuple order matters
con.commit()
print(cur.rowcount, "record(s) updated")
```

### DELETE with `%s`

```
rollno = 102
sql = "DELETE FROM students WHERE rollno = %s"
cur.execute(sql, (rollno,))  # Note: single value tuple needs a comma
con.commit()
print(cur.rowcount, "record(s) deleted")
```

### SELECT with `%s`

```
# Select records based on condition
city = "Delhi"
cur.execute("SELECT * FROM students WHERE city = %s", (city,))
rows = cur.fetchall()
```

```
for row in rows:
    print(row)
```

### Batch Insert with `executemany()`

```
# Insert multiple rows at once
sql = "INSERT INTO students (rollno, name, age, city) VALUES (%s, %s,
%s, %s)"
data = [
    (104, "Amit Kumar", 19, "Pune"),
    (105, "Sneha Patel", 20, "Chennai"),
    (106, "Rohan Das", 18, "Hyderabad")
]
cur.executemany(sql, data)  # executemany() for batch operations
con.commit()
print(cur.rowcount, "record(s) inserted")
```

## 5.2 Using `format()` Method

The `format()` method inserts values directly into the SQL string using `{}` placeholders.

> ⚠️ **Caution:** `format()` is **less secure** than `%s` because it makes your code vulnerable to SQL injection. It should be used **only** with trusted/static values.

### INSERT with `format()`

```
rollno = 107
name = "Priya Singh"
age = 21
city = "Jaipur"

# Using format() to insert values
cur.execute("INSERT INTO students VALUES ({}, '{}', {},
'{}')".format(rollno, name, age, city))
con.commit()
print(cur.rowcount, "record(s) inserted")
```

### UPDATE with `format()`

```
new_city = "Agra"
rollno = 101
cur.execute("UPDATE students SET city = '{}' WHERE rollno =
{}".format(new_city, rollno))
con.commit()
print(cur.rowcount, "record(s) updated")
```

### SELECT with `format()`

```
min_age = 18
max_age = 22
cur.execute("SELECT * FROM students WHERE age BETWEEN {} AND
{}".format(min_age, max_age))
rows = cur.fetchall()
for row in rows:
    print(row)
```

## 5.3 Comparison: `%s` vs `format()`

| Feature | `%s` Placeholder | `format()` Method |
|---|---|---|
| **Syntax** | `execute(sql, (val1, val2))` | `execute(sql.format(val1, val2))` |
| **Security** | ✅ **Secure** - prevents SQL injection | ❌ **Vulnerable** to SQL injection |
| **Recommended** | ✅ Yes (preferred method) | ❌ Only for trusted/static values |
| **Parameter Type** | Values passed as **tuple** | Values embedded using `{}` |
| **Batch Operations** | ✅ Supports `executemany()` | ❌ Does not support batch operations |

---

# 6. Complete Connectivity Application

A complete application that demonstrates **creating a database, establishing connection, and performing all CRUD operations** using parameterized queries.

```
import mysql.connector as sql

# ———————————— CLASS: StudentDB ————————————
class StudentDB:
    def __init__(self):
        try:
            # Connect without database to create if needed
            self.conn = sql.connect(
                host="localhost",
                user="root",
                passwd="mysql123"
            )
            self.cursor = self.conn.cursor()

            # Create database if it doesn't exist
            self.cursor.execute("CREATE DATABASE IF NOT EXISTS
school_db")
            print("✓ Database 'school_db' ready")

            # Switch to the database
            self.conn.database = "school_db"

            # Create table if it doesn't exist
            self.cursor.execute("""CREATE TABLE IF NOT EXISTS students (
                rollno INT PRIMARY KEY,
                name VARCHAR(30),
                age INT,
                city VARCHAR(20)
            )""")
            print("✓ Table 'students' ready")

        except sql.Error as e:
            print("❌ Error:", e)
            exit()

    # ——— INSERT using %s ———
    def insert_student(self, rollno, name, age, city):
        sql_query = "INSERT INTO students VALUES (%s, %s, %s, %s)"
        values = (rollno, name, age, city)
        self.cursor.execute(sql_query, values)
        self.conn.commit()
        print(self.cursor.rowcount, "record inserted")
```

```
    # ——— UPDATE using %s ———
    def update_student(self, rollno, new_city):
        sql_query = "UPDATE students SET city = %s WHERE rollno = %s"
        self.cursor.execute(sql_query, (new_city, rollno))
        self.conn.commit()
        print(self.cursor.rowcount, "record updated")

    # ——— DELETE using %s ———
    def delete_student(self, rollno):
        sql_query = "DELETE FROM students WHERE rollno = %s"
        self.cursor.execute(sql_query, (rollno,))
        self.conn.commit()
        print(self.cursor.rowcount, "record deleted")

    # ——— SELECT using %s ———
    def display_all(self):
        self.cursor.execute("SELECT * FROM students")
        rows = self.cursor.fetchall()
        print("\n--- Student Records ---")
        print("RollNo  Name              Age  City")
        print("-" * 40)
        for row in rows:
            print("%-6d %-15s %-6d %-10s" % (row[0], row[1], row[2],
row[3]))
        print("Total records:", self.cursor.rowcount)

    # ——— SELECT with condition using %s ———
    def display_by_city(self, city):
        self.cursor.execute("SELECT * FROM students WHERE city = %s",
(city,))
        rows = self.cursor.fetchall()
        print(f"\n--- Students from {city} ---")
        for row in rows:
            print("%-6d %-15s %-6d %-10s" % (row[0], row[1], row[2],
row[3]))

    # ——— INSERT using format() (demonstration) ———
    def insert_using_format(self, rollno, name, age, city):
        # Note: format() is less secure, use only with trusted values
        self.cursor.execute(
            "INSERT INTO students VALUES ({}, '{}', {},
'{}')".format(rollno, name, age, city)
```

```
        )
        self.conn.commit()
        print(self.cursor.rowcount, "record inserted using format()")

    # ——— CLOSE ———
    def close(self):
        self.cursor.close()
        self.conn.close()
        print("✓ Connection closed")

# ———————————— MAIN PROGRAM ————————————
db = StudentDB()

# Insert records using %s
db.insert_student(101, "Geeta Sharma", 20, "Kolkata")
db.insert_student(102, "Raj Kumar", 19, "Delhi")
db.insert_student(103, "Priya Singh", 21, "Mumbai")

# Insert using format() (demonstration)
db.insert_using_format(104, "Amit Patel", 22, "Chennai")

# Display all records
db.display_all()

# Update using %s
db.update_student(101, "Pune")

# Display updated records
db.display_all()

# Display by city using %s
db.display_by_city("Delhi")

# Delete using %s
db.delete_student(102)

# Final display
db.display_all()

# Close connection
db.close()
```

## 7. Summary Table

| Component | Syntax | Purpose |
|---|---|---|
| Import | `import mysql.connector as sql` | Load MySQL connector |
| Connect | `con = sql.connect(host, user, passwd)` | Connect without database |
| Connect (with DB) | `con = sql.connect(host, user, passwd, database)` | Connect to specific database |
| Create Database | `cur.execute("CREATE DATABASE IF NOT EXISTS db_name")` | Create new database |
| Switch Database | `con.database = "db_name"` | Switch to another database |
| Cursor | `cur = con.cursor()` | Create cursor for queries |
| Execute ( `%s` ) | `cur.execute(sql, (val1, val2))` | Secure parameterized query |
| Execute ( `format()` ) | `cur.execute(sql.format(val1, val2))` | Less secure - use cautiously |
| Batch Insert | `cur.executemany(sql, list_of_tuples)` | Insert multiple rows at once |
| fetchall() | `data = cur.fetchall()` | Fetch all rows → list of tuples |
| fetchone() | `row = cur.fetchone()` | Fetch one row → tuple |
| fetchmany(n) | `data = cur.fetchmany(n)` | Fetch n rows → list of tuples |
| rowcount | `cur.rowcount` | Number of rows affected/retrieved |
| commit() | `con.commit()` | Save DML changes permanently |
| close() | `cur.close(); con.close()` | Release resources |

## 8. Important Points for CBSE Exams

- `mysql.connector` is the module used for Python-MySQL connectivity.
- `connect()` – 3 compulsory parameters (host, user, password) and 1 optional (database).
- To **create a new database**, connect **without** the `database` parameter.
- `cursor()` creates a cursor with **NO parameters**.
- `execute()` runs SQL queries through the cursor.
- `%s` **placeholders** are the **recommended and secure** way to pass dynamic values.
- `format()` can be used but is **less secure** (vulnerable to SQL injection).
- `fetchall()` returns a **list of tuples**, `fetchone()` returns a **single tuple**.
- `rowcount` works with both SELECT and DML queries.
- `commit()` is **mandatory** for INSERT, UPDATE, DELETE operations.
- **Retrieving** = SQL → Cursor | **Fetching** = Cursor → Python.
- Always close cursor and connection to prevent resource leaks.

---

## 9. Common Errors and Solutions

| Error | Possible Cause | Solution |
|---|---|---|
| Access denied | Wrong username/password | Check credentials |
| Unknown database | Database doesn't exist | Create database first or check name |
| Not connected | No connection established | Check if `connect()` executed successfully |
| No result set | Trying to fetch without SELECT | Execute SELECT before fetching |
| Unread result found | Forgetting to fetch before next query | Fetch all results before executing next query |
| SQL injection vulnerability | Using `format()` with user input | Use `%s` placeholders instead |

---

## 10. Quick Reference Card

🔹 **Create Database:**

```
con = sql.connect(host="localhost", user="root", passwd="mysql123")
cur.execute("CREATE DATABASE IF NOT EXISTS school_db")
con.database = "school_db"
```

🔹 **INSERT using %s:**

```
sql = "INSERT INTO students VALUES (%s, %s, %s, %s)"
cur.execute(sql, (101, "Geeta", 20, "Delhi"))
con.commit()
```

🔹 **INSERT using format():**

```
cur.execute("INSERT INTO students VALUES ({}, '{}', {},
'{}')".format(101, "Geeta", 20, "Delhi"))
con.commit()
```

🔹 **SELECT with %s:**

```
cur.execute("SELECT * FROM students WHERE city = %s", ("Delhi",))
rows = cur.fetchall()
```

🔹 **UPDATE with %s:**

```
cur.execute("UPDATE students SET city = %s WHERE rollno = %s",
("Mumbai", 101))
con.commit()
```

🔹 **DELETE with %s:**

```
cur.execute("DELETE FROM students WHERE rollno = %s", (101,))
con.commit()
```